

Phase 7 Report

Kubernetes Workload Analyzer — production hardening, in-cluster deploy

Phase status	DONE (2026-09-26)
Report date	2026-09-26
Repository	github.com/ISHANT57/kubernetes-workload-analyzer
State of main	15382bc , merged as 75c112d (PR #12)
Image	gcr.io/distroless/static-debian12:nonroot , 45.7 MB, non-root, no shell
Written for	the project owner, as a study and reference document

Evidence labels used throughout. **MEASURED** a command was run on this machine and the number is its output · **VERIFIED** checked against upstream documentation, source or a registry · **ESTIMATED** a projection, not measured · **UNVERIFIED** not yet tested; a test is named.

Contents

1. What Phase 7 was for
2. Multi-stage Dockerfile
3. Dedicated namespace and hardened Deployment
4. A real distroless gotcha, hit and fixed
5. Resources sized from measurement, not guessed
6. D-007 RBAC re-verified live with a real token
7. A methodology bug: the Operator reverted a direct scale-down
8. The real Prometheus outage/recovery test
9. Git and GitHub workflow
10. Current state, operating and rollback
11. Open items and next phase

1. What Phase 7 was for

Phases 3-6 all ran on the host, against the admin kubeconfig. Phase 7 is where the analyzer actually becomes a Kubernetes workload: containerized, deployed with its own scoped ServiceAccount, hardened, and proven to fail and recover correctly under a real, deliberate outage — not a simulated one. Done condition (`PHASES.md`): "Failure tests pass (Prometheus down, partial query failure, duplicate run, insufficient history)".

2. Multi-stage Dockerfile

backend/Dockerfile: `node:24-alpine` builds the frontend, then `golang:1.27-alpine` builds the Go binary (`CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w"`), then both are copied into a final `gcr.io/distroless/static-debian12:nonroot` stage — no shell, no package manager, no build tools in the shipped image. Built image: **45.7 MB**.

3. Dedicated namespace and hardened Deployment

`deploy/rbac/analyzer.yaml` gained a dedicated `analyzer` namespace (ServiceAccount moved out of `default`); the D-007 ClusterRole/ClusterRoleBinding content is unchanged from Phase 3.

`deploy/analyzer/deployment.yaml` (new): Deployment + Service with:

- `runAsNonRoot: true`, explicit `runAsUser / runAsGroup: 65532`
- `allowPrivilegeEscalation: false`
- `readOnlyRootFilesystem: true`
- all Linux capabilities dropped
- `seccompProfile: RuntimeDefault`
- no `kubeconfig` mounted — `rest.InClusterConfig()` picks up the ServiceAccount token automatically
- `PROMETHEUS_URL` pointed at the in-cluster Service DNS name instead of `localhost`

4. A real distroless gotcha, hit and fixed

CreateContainerConfigError: container has runAsNonRoot and image has non-numeric user (nonroot), cannot verify user is non-root. distroless's nonroot image sets USER nonroot — a name, not a UID — which the kubelet cannot resolve to satisfy runAsNonRoot: true without an explicit numeric runAsUser/ runAsGroup. Fixed by adding runAsUser: 65532, runAsGroup: 65532 to the pod's securityContext.

5. Resources sized from measurement, not guessed

`kubectl top` against the running pod showed idle usage around **~1m CPU / 10Mi memory**, peaking to **~18m CPU** during an analysis run (`docs/architecture.md`, "Where it runs"). The Deployment's `requests` (`cpu: 25m, memory: 32Mi`) and `memory limit` (`128Mi`) were set from that measurement with headroom — the analyzer's own "measure, don't guess" advice applied to itself. An initial guess of `20m/48Mi` was replaced once the real numbers were in hand.

6. D-007 RBAC re-verified live with a real token

Not re-simulated — a real bearer token was pulled from the actual in-cluster ServiceAccount and used from an isolated kubeconfig:

```
kubectl -n analyzer create token analyzer
KUBECONFIG=/dev/null kubectl --server=<api-server> --token=<token> --insecure-skip-tls-verify=true \
  auth can-i list pods -A      # no
  auth can-i list secrets -A   # no
  auth can-i list deployments -A # yes
  auth can-i create deployments -A # no
  auth whoami                  # system:serviceaccount:analyzer:analyzer
```

`kubectl auth whoami` was used to confirm real identity, not `--as=` impersonation, which would not have proven the token itself carries these restrictions. [docs/threat-model.md](#)'s ServiceAccount-token-leak mitigation row cites this exact verification.

7. A methodology bug: the Operator reverted a direct scale-down

The first attempt to simulate a Prometheus outage scaled the StatefulSet directly (`kubectl scale statefulset ... --replicas=0`). The Prometheus Operator silently reverted it within ~52 seconds — it reconciles from the Prometheus custom resource's desired state, not from manual StatefulSet edits — producing a false-negative test result (`status: complete` when Prometheus had, in fact, already been restored by the operator before the check ran). **Fixed** by patching the Prometheus CR's own `spec.replicas` instead, which the operator honors as the real desired state:

```
kubectl -n monitoring patch prometheus kps-kube-prometheus-stack-prometheus --type merge -p '{"spec":{"replicas":0}}'
```

8. The real Prometheus outage/recovery test

Outage (Prometheus CR scaled to 0):

```
{
  "status": "partial",
  "workloads_seen": 17,
  "findings_count": 3,
  "query_errors": [{"source": "prometheus",
    "message": "...dial tcp 10.96.240.82:9090: connect: connection refused"}]
}
```

/readyz → **503**. findings_count stayed at its last-good value (sticky), never dropped to 0.

Recovery (CR restored to replicas: 1): once the Prometheus pod became Ready, the analyzer's **next scheduled cycle** recovered on its own — status: complete, /readyz → **200** — with no restart of the analyzer itself needed.

9. Git and GitHub workflow

Branch `phase-7` → commit `15382bc` ("feat(phase-7): production hardening, in-cluster deploy") → PR #12 → merged as `75c112d` → branch deleted.

10. Current state, operating and rollback

```
docker build -f backend/Dockerfile -t k8s-workload-analyzer:local .
kind load docker-image k8s-workload-analyzer:local --name workload-analyzer
kubectl apply -f deploy/rbac/analyzer.yaml
kubectl apply -f deploy/analyzer/deployment.yaml
kubectl -n analyzer rollout status deploy/analyzer
kubectl -n analyzer port-forward svc/analyzer 8080:8080
```

Rollback: `kubectl delete -f deploy/analyzer/deployment.yaml -f deploy/rbac/analyzer.yaml`.

11. Open items and next phase

- No scripted end-to-end demo yet — Phase 8.
- No formal load test / performance benchmark document existed at the time this phase closed — added afterward as a follow-up artifact, see [docs/performance.md](#).